

Universidad Nacional Autonoma de México.

Facultad de Estudios Superiores Acatlán.

Tecnicas Estadisticas y Minería de Datos

Modulo IV

Practica: Analisis de Datos_Continuacion

Profesor: JAVIER ROSAS HERNÁNDEZ \

Grupo - 35
de Junio del 2025

22

Datos

Recordemos que en la **Practica #2**; habiamos realizado un analisis de un conjunto de datos que habiamos desconocido tenemos, las siguiente actividad que nosn ayuda a resolver algunas dudas de dicha practica.

Con los archivos siguientes:

- **Big5raw.csv** (Datos en crudo)
- **Codebook** (Descripción de los datos)

Antes de comenzar hay que establecer algunas paqueterias necesarios para realizar cada uno de los incisos y de paso el exporta la informacion para su previa manipulacion breve.

```
import numpy as np      # Manipulacion de estructuras matematicas
import pandas as pd     # Manipulacion de base de datos en tablas
import seaborn as sns   # Creacion de graficas estadisticas
import matplotlib.pyplot as plt  # Creacion de las graficas descriptivas en general.
```

```
#===== importamos Los datos =====#
```

```
tabla = pd.read_csv('Big5raw.csv', header = 0)
```

```
# Vemos un pequeño visualización de los datos
tabla.head(12)
```

	E1	E2	E3	E4	E5	E6	E7	E8	E9	E10	...	O8	O9	O10	race	age	engnat	gender	hand	s
0	4	2	5	2	5	1	4	3	5	1	...	2	5	5	3	53.0	1.0	1	1	
1	2	2	3	3	3	3	1	5	1	5	...	1	3	2	13	46.0	1.0	2	1	
2	5	1	1	4	5	1	1	5	5	1	...	5	5	5	1	14.0	2.0	2	1	
3	2	5	2	4	3	4	3	4	4	5	...	2	5	5	3	19.0	2.0	2	1	
4	3	1	3	3	3	1	3	1	3	5	...	1	5	3	11	25.0	2.0	2	1	
5	1	5	2	4	1	3	2	4	1	5	...	4	5	3	13	31.0	1.0	2	1	
6	5	1	5	1	5	1	5	4	4	1	...	3	3	4	5	20.0	1.0	2	1	
7	4	3	5	3	5	1	4	3	4	3	...	3	2	5	4	23.0	2.0	1	1	
8	3	1	5	1	5	1	5	2	5	3	...	3	4	5	5	39.0	1.0	2	3	
9	1	4	2	0	2	4	1	4	1	5	...	3	4	4	3	18.0	1.0	2	1	
10	1	5	2	5	1	4	1	4	1	5	...	5	4	5	3	17.0	2.0	2	1	
11	3	3	5	3	3	3	2	4	3	3	...	3	4	5	13	15.0	2.0	1	1	

12 rows × 57 columns



Una de las diferencias que tenemos con los datos anteriores es con las descripción de los datos y el título de algunas V.A.s (en especial la últimas que pasaron de Letras a texto que nos da incisión de los datos).

En cuanto a la descripción tenemos el siguiente resumen del archivo **Codebook**:

Es una **prueba de personalidad interactiva en línea**; los datos son utilizados para una análisis posterior consta de una encuesta de las cuales califican en escala de 1 — 5 (donde 1 es el valor con menor valoración y el 5 en máximo) para la mayoría de las Columnas, las cuales son:

Grupo E	Grupo E
E1: Soy el alma de la fiesta.	E6: Tengo poco que decir.
E2: No hablo mucho.	E7: Hablo con mucha gente diferente en las fiestas
E3: Me siento cómodo con la gente.	E8: No me gusta llamar la atención.
E4: Me mantengo en un segundo plano.	E9: No me importa ser el centro de atención.
E5: Inicié conversaciones.	E10: Soy tranquilo con desconocidos.
Grupo N	Grupo N
N1: Me estreso fácilmente.	N6: Me enoja con facilidad.

Grupo N		Grupo N	
N2: Estoy relajado la mayor parte del tiempo.		N7: Cambio mucho de humor.	
N3: Me preocupo por las cosas.		N8: Tengo cambios de humor frecuentes.	
N4: Rara vez me siento triste.		N9: Me irrito fácilmente.	
N5: Me altero con facilidad.		N10: A menudo me siento triste	
Grupo A		Grupo A	
A1: Me preocupo poco por los demás.		A6: Soy de corazón blando.	
A2: Me intereso por la gente.		A7: No me interesan mucho los demás.	
A3: Insulto a la gente.		A8: Dedico tiempo a los demás.	
A4: Me solidarizo con los sentimientos de los demás.		A9: Siento las emociones de los demás.	
A5: No me interesan los problemas de los demás.		A10: Hago que la gente se sienta cómoda.	
Grupo C		Grupo C	
C1: Siempre estoy preparado.		C6: A menudo olvido poner las cosas en su sitio.	
C2: Dejo mis pertenencias por todas partes.		C7: Me gusta el orden.	
C3: Presto atención a los detalles.		C8: Eludo mis obligaciones.	
C4: Hago un desastre.		C9: Sigo un horario.	
C5: Hago las tareas enseguida.		C10: Soy exigente en mi trabajo.	
Grupo O		Grupo O	
O1: Tengo un vocabulario rico.		O6: No tengo buena imaginación.	
O2: Me cuesta entender ideas abstractas.		O7: Comprendo las cosas con rapidez.	
O3: Tengo una imaginación vívida.		O8: Uso palabras difíciles.	
O4: No me interesan las ideas abstractas.		O9: Dedico tiempo a reflexionar.	
O5: Tengo ideas excelentes.		O10: Tengo muchas ideas.	

Donde 0 indica que no hay registro alguno en dicha pregunta.

Recalcando que estas preguntas estan agrupadas por el tipo de tema que abarcan. Pasando al resto de las columnas tenemos que:

- **A = race:** pregunta que se responde seleccionado de un menú desplegable.
 - 1 = Raza mixta,
 - 2 = Ártico (siberiano, esquimal)

- 3 = Caucásico (europeo)
 - 4 = Caucásico (indio)
 - 5 = Caucásico (Oriente Medio)
 - 6 = Caucásico (norteafricano, otros)
 - 7 = Indígena australiano
 - 8 = Nativo americano
 - 9 = Noreste asiático (mongol, tibetano, coreano, japonés, etc.)
 - 10 = Pacífico (polinesio, micronesio, etc.)
 - 11 = Sureste asiático (chino, tailandés, malayo, filipino, etc.)
 - 12 = África occidental, bosquimano, etíope,
 - 13 = Otro
 - 0 = falta de contestar
- **B = Age:** Introducida como texto (no se registró a las personas menores de 13 años).
 - **C = Engnat:** Respuesta a "¿Es el inglés su lengua materna?" con los valores
 - 1 = sí
 - 2 = no
 - 0 = falta
 - **D = Gender:** Se selecciona de un menú desplegable con 4 valores
 - 1 = Hombre
 - 2 = Mujer
 - 3 = Otra
 - 0 = omitido
 - **E = Hand:** Respuesta a la pregunta "¿Con qué mano escribes?"
 - 1 = Derecha
 - 2 = Izquierda
 - 3 = Ambas
 - 0 = omitido

Algunos valores se calcularon a partir de información técnica:

- **E = Source:** Indicar de como llegaron a realizar este encuesta.
 - 1 = Otra página del sitio web de la prueba
 - 2 = Google
 - 3 = Facebook
 - 4 = Cualquier URL con ".edu" en su nombre de dominio (p. ej., xxx.edu, xxx.edu.au)
 - 6 = Otra fuente o no se proporcionó el HTTP Referer.
- **G = Country:** Ubicación técnica del participante. Código ISO del país.

Preguntas

Pregunta 1:

Conociendo la descripción de los datos; indica 3 ajustes a realizar en el análisis descriptivo de datos de *la Practica 2.

Solucion:

Resumiendo las acciones que se realizaron en las parte: **Analisis de Datos y Estadistica Descriptiva** se vio detalles como:

- El numero de V.A.s y registro
- Los tipos de datos en los registros
- Una tipo de agrupacion en las V.A.s
- las diferencia entras la columnas alfabetica de las demas
- Algunos terminos de estadisticos descriptiva (*descartando a la columna G por contener datos del tipo caracter*)

Revisando los tipos de datos de la base de datos tenemos que:

```
tabla.info(verbose = True)
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 19723 entries, 0 to 19722

Data columns (total 57 columns):

#	Column	Non-Null	Count	Dtype
0	E1	19723	non-null	int64
1	E2	19723	non-null	int64
2	E3	19723	non-null	int64
3	E4	19723	non-null	int64
4	E5	19723	non-null	int64
5	E6	19723	non-null	int64
6	E7	19723	non-null	int64
7	E8	19723	non-null	int64
8	E9	19723	non-null	int64
9	E10	19723	non-null	int64
10	N1	19723	non-null	int64
11	N2	19723	non-null	int64
12	N3	19723	non-null	int64
13	N4	19723	non-null	int64
14	N5	19723	non-null	int64
15	N6	19723	non-null	int64
16	N7	19723	non-null	int64
17	N8	19723	non-null	int64
18	N9	19723	non-null	int64
19	N10	19723	non-null	int64
20	A1	19723	non-null	int64
21	A2	19723	non-null	int64
22	A3	19723	non-null	int64
23	A4	19723	non-null	int64
24	A5	19723	non-null	int64
25	A6	19723	non-null	int64
26	A7	19723	non-null	int64
27	A8	19723	non-null	int64
28	A9	19723	non-null	int64
29	A10	19723	non-null	int64
30	C1	19723	non-null	int64
31	C2	19723	non-null	int64
32	C3	19723	non-null	int64
33	C4	19723	non-null	int64
34	C5	19723	non-null	int64
35	C6	19723	non-null	int64
36	C7	19723	non-null	int64
37	C8	19723	non-null	int64
38	C9	19723	non-null	int64
39	C10	19723	non-null	int64
40	O1	19723	non-null	int64
41	O2	19723	non-null	int64
42	O3	19723	non-null	int64
43	O4	19723	non-null	int64
44	O5	19723	non-null	int64
45	O6	19723	non-null	int64
46	O7	19723	non-null	int64
47	O8	19723	non-null	int64
48	O9	19723	non-null	int64
49	O10	19723	non-null	int64
50	race	19723	non-null	int64

```
51 age      19722 non-null Int64
52 engnat   19722 non-null Int64
53 gender   19723 non-null int64
54 hand     19723 non-null int64
55 source   19723 non-null int64
56 country  19711 non-null object
dtypes: Int64(2), int64(54), object(1)
memory usage: 8.6+ MB
```

La mayoría de los datos son numericos y con el contexto que tenemos sobre ellos.

1. El primer cambio seria el convertir todo los datos en cada una de las columnas en valores enteros.

Estos cambios es por el hecho de que este valores cuantifican las preguntas en un rango de valores discretos en general y no se hace utilizacion de valores continuos ya que no tendria sentido alguno...

Para esto lo aplicamos en las columnas:

age: tiene datos en formato de texto lo cual esta informacion tiene mas aportacion si es numerico entero

```
#Ignoramos Los errores de datos
tabla['age'] = pd.to_numeric(tabla['age'], errors='coerce').astype('Int64')
```

engnat: Si es para responder el nivel de ingles, lo mas conveniente es que sean numerico como dato ordinal.

```
#Ignoramos Los errores de datos
tabla['engnat'] = pd.to_numeric(tabla['engnat'], errors='coerce').astype('Int64')
```

country: Al ser datos de tipo caracter tenemos que reclasificarlo *(ya que si en la mayoría de las columnas realizaron este reclasifica, esta fue la unica que se aplico la transformacion)*.

Pero hay un problema, es el hecho de que no podemos asignarle un numero a cada **ISO CODE** ya que la lista de valores son muy abundantes y no todo los pais esta presente en la encuestas.

Por lo que el siguiente cambio para los datos descriptivos sera:

2. El segundo cambio seria el convertir los datos de la columna country (datos nominales) a una serie de columnas del tipo binario

Esto nos dara un aumento en el tamano de las columnas, ya que si observamos los valores unicos que tiene:

```
# Valores unicos de la columna "Country"
holis = tabla['country'].unique()
holis
```

```
array(['US', 'PK', 'RO', 'IN', 'IT', 'FR', 'GB', 'LT', 'CA', 'GR', 'DK',
      'MX', 'SE', 'JO', 'NO', 'IE', 'NZ', 'FI', 'BE', 'HR', 'ZA', 'PL',
      'BR', 'SK', 'SA', 'PH', 'AU', 'JP', 'ID', 'MY', 'KW', 'OM', 'TR',
      'SG', 'IL', 'DE', 'AE', 'BT', 'AR', 'RS', 'GH', 'NL', 'MU', 'KE',
      '(nu', 'AT', 'TZ', 'ES', nan, 'HU', 'RU', 'UG', 'PT', 'HK', 'CO',
      'CN', 'BH', 'ME', 'BZ', 'SI', 'CZ', 'CH', 'LK', 'EG', 'TH', 'BG',
      'AZ', 'PR', 'NP', 'FJ', 'CM', 'BD', 'VN', 'LB', 'PW', 'MA', 'IR',
      'EE', 'JM', 'UA', 'MM', 'A1', 'KR', 'CY', 'JE', 'QA', 'MK', 'VE',
      'IS', 'NG', 'ZW', 'CR', 'CL', 'SY', 'DO', 'AL', 'SV', 'USA', 'TT',
      'EU', 'BM', 'HT', 'TW', 'MZ', 'BW', 'BF', 'TN', 'IM', 'BA', 'KZ',
      'GG', 'SD', 'HN', 'GE', 'KH', 'PG', 'MV', 'A2', 'RW', 'LS', 'PE',
      'BB', 'LA', 'BO', 'VC', 'PA', 'BN', 'AG', 'LV', 'MP', 'EC', 'MT',
      'GP', 'AS', 'AP', 'FO', 'NI', 'GU', 'DZ', 'BS', 'SR', 'GY', 'LY',
      'IQ', 'GT', 'VI', 'KY', 'MN', 'ET', 'AO', 'UY', 'GD', 'MR', 'USS',
      'CV', 'MW', 'ZM', 'PY', 'UZ', 'MXN', 'TC', 'KG'], dtype=object)
```

```
# imprimos el numero de valores distintos
print(f"Tenemos un total de: {len(holis)} paises que se registraron de la encuesta" )
```

Tenemos un total de: 162 paises que se registraron de la encuesta

Pasando a la transformacion de los datos...

```
# La funcion "Dummies" crea nuevas columnas en base a los valores unicos y con ello podemos aln
tabla_new = pd.get_dummies(tabla, columns = ['country'])

# Visualizamos la tabla
tabla_new.head(12)
```

	E1	E2	E3	E4	E5	E6	E7	E8	E9	E10	...	country_USS	country_UY	country_UZ	country_VC
0	4	2	5	2	5	1	4	3	5	1	...	False	False	False	False
1	2	2	3	3	3	3	1	5	1	5	...	False	False	False	False
2	5	1	1	4	5	1	1	5	5	1	...	False	False	False	False
3	2	5	2	4	3	4	3	4	4	5	...	False	False	False	False
4	3	1	3	3	3	1	3	1	3	5	...	False	False	False	False
5	1	5	2	4	1	3	2	4	1	5	...	False	False	False	False
6	5	1	5	1	5	1	5	4	4	1	...	False	False	False	False
7	4	3	5	3	5	1	4	3	4	3	...	False	False	False	False
8	3	1	5	1	5	1	5	2	5	3	...	False	False	False	False
9	1	4	2	0	2	4	1	4	1	5	...	False	False	False	False
10	1	5	2	5	1	4	1	4	1	5	...	False	False	False	False
11	3	3	5	3	3	3	2	4	3	3	...	False	False	False	False

12 rows × 217 columns

Con esto accion, sera mas preciso la informacion que deseamos analizar, pero hay que convertir los datos en numeros enteros...

```
# Por el bucle "for" realizamos los cambios, pero a partir de las nuevas columnas (en la posicion)
for i in tabla_new.columns[57:]:
    tabla_new[i] = pd.to_numeric(tabla_new[i], errors='coerce').astype('Int64')

# Visualizamos la tabla
tabla_new.head(12)
```

	E1	E2	E3	E4	E5	E6	E7	E8	E9	E10	...	country_USS	country_UY	country_UZ	country_VC
0	4	2	5	2	5	1	4	3	5	1	...	0	0	0	0
1	2	2	3	3	3	3	1	5	1	5	...	0	0	0	0
2	5	1	1	4	5	1	1	5	5	1	...	0	0	0	0
3	2	5	2	4	3	4	3	4	4	5	...	0	0	0	0
4	3	1	3	3	3	1	3	1	3	5	...	0	0	0	0
5	1	5	2	4	1	3	2	4	1	5	...	0	0	0	0
6	5	1	5	1	5	1	5	4	4	1	...	0	0	0	0
7	4	3	5	3	5	1	4	3	4	3	...	0	0	0	0
8	3	1	5	1	5	1	5	2	5	3	...	0	0	0	0
9	1	4	2	0	2	4	1	4	1	5	...	0	0	0	0
10	1	5	2	5	1	4	1	4	1	5	...	0	0	0	0
11	3	3	5	3	3	3	2	4	3	3	...	0	0	0	0

12 rows × 217 columns



Para corroborar si los cambios que hemos realizado se haya aplicado satisfactoriamente, lo veremos por el siguiente codigo...

```
tabla_new.info(verbose = True)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 19723 entries, 0 to 19722
```

```
Data columns (total 217 columns):
```

#	Column	Dtype
0	E1	int64
1	E2	int64
2	E3	int64
3	E4	int64
4	E5	int64
5	E6	int64
6	E7	int64
7	E8	int64
8	E9	int64
9	E10	int64
10	N1	int64
11	N2	int64
12	N3	int64
13	N4	int64
14	N5	int64
15	N6	int64
16	N7	int64
17	N8	int64
18	N9	int64
19	N10	int64
20	A1	int64
21	A2	int64
22	A3	int64
23	A4	int64
24	A5	int64
25	A6	int64
26	A7	int64
27	A8	int64
28	A9	int64
29	A10	int64
30	C1	int64
31	C2	int64
32	C3	int64
33	C4	int64
34	C5	int64
35	C6	int64
36	C7	int64
37	C8	int64
38	C9	int64
39	C10	int64
40	O1	int64
41	O2	int64
42	O3	int64
43	O4	int64
44	O5	int64
45	O6	int64
46	O7	int64
47	O8	int64
48	O9	int64
49	O10	int64
50	race	int64

51	age	Int64
52	engnat	Int64
53	gender	int64
54	hand	int64
55	source	int64
56	country_(nu	bool
57	country_A1	Int64
58	country_A2	Int64
59	country_AE	Int64
60	country_AG	Int64
61	country_AL	Int64
62	country_AO	Int64
63	country_AP	Int64
64	country_AR	Int64
65	country_AS	Int64
66	country_AT	Int64
67	country_AU	Int64
68	country_AZ	Int64
69	country_BA	Int64
70	country_BB	Int64
71	country_BD	Int64
72	country_BE	Int64
73	country_BF	Int64
74	country_BG	Int64
75	country_BH	Int64
76	country_BM	Int64
77	country_BN	Int64
78	country_BO	Int64
79	country_BR	Int64
80	country_BS	Int64
81	country_BT	Int64
82	country_BW	Int64
83	country_BZ	Int64
84	country_CA	Int64
85	country_CH	Int64
86	country_CL	Int64
87	country_CM	Int64
88	country_CN	Int64
89	country_CO	Int64
90	country_CR	Int64
91	country_CV	Int64
92	country_CY	Int64
93	country_CZ	Int64
94	country_DE	Int64
95	country_DK	Int64
96	country_DO	Int64
97	country_DZ	Int64
98	country_EC	Int64
99	country_EE	Int64
100	country_EG	Int64
101	country_ES	Int64
102	country_ET	Int64
103	country_EU	Int64
104	country_FI	Int64
105	country_FJ	Int64
106	country_FO	Int64

107	country_FR	Int64
108	country_GB	Int64
109	country_GD	Int64
110	country_GE	Int64
111	country_GG	Int64
112	country_GH	Int64
113	country_GP	Int64
114	country_GR	Int64
115	country_GT	Int64
116	country_GU	Int64
117	country_GY	Int64
118	country_HK	Int64
119	country_HN	Int64
120	country_HR	Int64
121	country_HT	Int64
122	country_HU	Int64
123	country_ID	Int64
124	country_IE	Int64
125	country_IL	Int64
126	country_IM	Int64
127	country_IN	Int64
128	country_IQ	Int64
129	country_IR	Int64
130	country_IS	Int64
131	country_IT	Int64
132	country_JE	Int64
133	country_JM	Int64
134	country_JO	Int64
135	country_JP	Int64
136	country_KE	Int64
137	country_KG	Int64
138	country_KH	Int64
139	country_KR	Int64
140	country_KW	Int64
141	country_KY	Int64
142	country_KZ	Int64
143	country_LA	Int64
144	country_LB	Int64
145	country_LK	Int64
146	country_LS	Int64
147	country_LT	Int64
148	country_LV	Int64
149	country_LY	Int64
150	country_MA	Int64
151	country_ME	Int64
152	country_MK	Int64
153	country_MM	Int64
154	country_MN	Int64
155	country_MP	Int64
156	country_MR	Int64
157	country_MT	Int64
158	country_MU	Int64
159	country_MV	Int64
160	country_MW	Int64
161	country_MX	Int64
162	country_MXN	Int64

163	country_MY	Int64
164	country_MZ	Int64
165	country_NG	Int64
166	country_NI	Int64
167	country_NL	Int64
168	country_NO	Int64
169	country_NP	Int64
170	country_NZ	Int64
171	country_OM	Int64
172	country_PA	Int64
173	country_PE	Int64
174	country_PG	Int64
175	country_PH	Int64
176	country_PK	Int64
177	country_PL	Int64
178	country_PR	Int64
179	country_PT	Int64
180	country_PW	Int64
181	country_PY	Int64
182	country_QA	Int64
183	country_RO	Int64
184	country_RS	Int64
185	country_RU	Int64
186	country_RW	Int64
187	country_SA	Int64
188	country_SD	Int64
189	country_SE	Int64
190	country_SG	Int64
191	country_SI	Int64
192	country_SK	Int64
193	country_SR	Int64
194	country_SV	Int64
195	country_SY	Int64
196	country_TC	Int64
197	country_TH	Int64
198	country_TN	Int64
199	country_TR	Int64
200	country_TT	Int64
201	country_TW	Int64
202	country_TZ	Int64
203	country_UA	Int64
204	country_UG	Int64
205	country_US	Int64
206	country_USA	Int64
207	country_USS	Int64
208	country_UY	Int64
209	country_UZ	Int64
210	country_VC	Int64
211	country_VE	Int64
212	country_VI	Int64
213	country_VN	Int64
214	country_ZA	Int64
215	country_ZM	Int64
216	country_ZW	Int64

dtypes: Int64(162), bool(1), int64(54)
memory usage: 35.6 MB

Todo las V.A.s tiene ya tiene los datos en numeros enteros pero solo uno no; **country_(nu** ya que tiene la funcion destacar el registro marco el pais o no (*por eso es booleano*)

```
# Veamos Los valores que maneja la columna "country_(nu"
bro = tabla_new['country_(nu)'].unique()
bro
```

```
array([False,  True])
```

Ya por ultimo, recordemos que las preguntas que se realizaron en la encuestas eran calificadas en 1 – 5 y algunas de las respuesta tiene valores fuera de dicho rango y por ello debemos de cambiar dichos valores; pero como ???

Con la siguiente regla:

- Si el valor es mayor que 5, suponemos que los respuesta a esa pregunta es es totalmente satisfactoria y por ende sera marcaremos en 5.
- Todo lo contrario, si el valor es menor que cero
- Los valores Nulls los dejamos asi como esta (*los que son cero esta exento a esta regla*).

Las columnas que infringen el rango de valores para responder son las siguiente: **E10, N10, C8, 03 Y 09** (*por el analisis de datos realizado en la practica anterior*)

```
# Almacenamos Los nombre de Las columnas
columnas_corregir = ['E10', 'N10', 'C8', '03', '09']

# Checa cada uno de Los registro y plica la regla que hemos dicho
for col in columnas_corregir:
    tabla_new[col] = np.where(tabla_new[col] > 5, 5, tabla_new[col]) # valores > 5 → 5
    tabla_new[col] = np.where(tabla_new[col] < 0, 1, tabla_new[col]) # valores < 0 → 1
```

Veamos si las columnas tiene los rangos que estipula la descripcion...

```
# Para el caso de Las columnas que son semejantes
for i in range(len(tabla_new[0:50])):
    print(f"La columna {tabla_new.columns[i]} tiene estos valores: {tabla_new.iloc[:, i].unique()})
```

La columna E1 tiene estos valores: [4 2 5 3 1 0]
La columna E2 tiene estos valores: [2 1 5 3 4 0]
La columna E3 tiene estos valores: [5 3 1 2 4 0]
La columna E4 tiene estos valores: [2 3 4 1 0 5]
La columna E5 tiene estos valores: [5 3 1 2 4 0]
La columna E6 tiene estos valores: [1 3 4 2 5 0]
La columna E7 tiene estos valores: [4 1 3 2 5 0]
La columna E8 tiene estos valores: [3 5 4 1 2 0]
La columna E9 tiene estos valores: [5 1 4 3 2 0]
La columna E10 tiene estos valores: [1 5 3 4 2 0]
La columna N1 tiene estos valores: [1 2 5 3 4 0]
La columna N2 tiene estos valores: [5 3 1 4 2 0]
La columna N3 tiene estos valores: [2 4 5 3 1 0]
La columna N4 tiene estos valores: [5 2 4 3 1 0]
La columna N5 tiene estos valores: [1 3 5 4 2 0]
La columna N6 tiene estos valores: [1 4 5 3 2 0]
La columna N7 tiene estos valores: [1 3 5 4 2 0]
La columna N8 tiene estos valores: [1 2 5 3 4 0]
La columna N9 tiene estos valores: [1 2 5 4 3 0]
La columna N10 tiene estos valores: [1 4 5 2 3 0]
La columna A1 tiene estos valores: [1 5 2 4 3 0]
La columna A2 tiene estos valores: [5 3 1 2 4 0]
La columna A3 tiene estos valores: [1 3 5 4 2 0]
La columna A4 tiene estos valores: [5 4 2 1 3 0]
La columna A5 tiene estos valores: [2 4 1 3 5 0]
La columna A6 tiene estos valores: [3 4 5 2 1 0]
La columna A7 tiene estos valores: [1 2 3 5 4 0]
La columna A8 tiene estos valores: [5 3 4 2 1 0]
La columna A9 tiene estos valores: [4 5 3 1 2 0]
La columna A10 tiene estos valores: [5 3 4 2 1 0]
La columna C1 tiene estos valores: [4 3 2 5 1 0]
La columna C2 tiene estos valores: [1 3 5 4 2 0]
La columna C3 tiene estos valores: [5 3 4 2 1 0]
La columna C4 tiene estos valores: [1 2 5 3 4 0]
La columna C5 tiene estos valores: [5 3 1 4 2 0]
La columna C6 tiene estos valores: [1 4 3 2 5 0]
La columna C7 tiene estos valores: [4 5 1 3 2 0]
La columna C8 tiene estos valores: [1 4 3 2 5 0]
La columna C9 tiene estos valores: [4 5 2 3 1 0]
La columna C10 tiene estos valores: [5 4 3 2 1 0]
La columna O1 tiene estos valores: [4 3 5 2 1 0]
La columna O2 tiene estos valores: [1 3 5 2 4 0]
La columna O3 tiene estos valores: [3 5 1 4 2 0]
La columna O4 tiene estos valores: [1 3 2 4 5 0]
La columna O5 tiene estos valores: [5 2 4 3 1 0]
La columna O6 tiene estos valores: [1 3 2 5 4 0]
La columna O7 tiene estos valores: [4 3 5 2 1 0]
La columna O8 tiene estos valores: [2 1 5 4 3 0]
La columna O9 tiene estos valores: [5 3 2 4 1 0]
La columna O10 tiene estos valores: [5 2 3 4 1 0]

Pregunta 2:

Identificar el top 5 de variables de personalidad que consideres más representativas. Justificar.

Solucion:

Como habiamos visto; las preguntas al parecer este agrupadas en columnas de 10 y que abarca un tema en concreto de la personalidad del sujeto que contesto la encuesta.

Como son 5 grupos, es claro que las 5 V.A.s sera seleccionada una de cada grupo. Pero el datalle recaí en que características nos baseremos para escogerlas???

Para eso creo que lo mas convenientes sera aquella que contenga **una buena correccion con el resto de las columnas por grupo....**

La razon de esto es que seguramente esrta preguntas sera las que mas represente el tema del cual se quiere indagar de los encuestados. Y en resumente las 5 sera las siguientes:

- E7
- N8
- A9
- C10
- O1

Veamos el proceso de las matrices...

Grupo E

```
# Creamos La matriz de Correlacion con La funcion spearman
corSp_E = tabla_new.iloc[:, :10].corr( method = 'spearman')
```

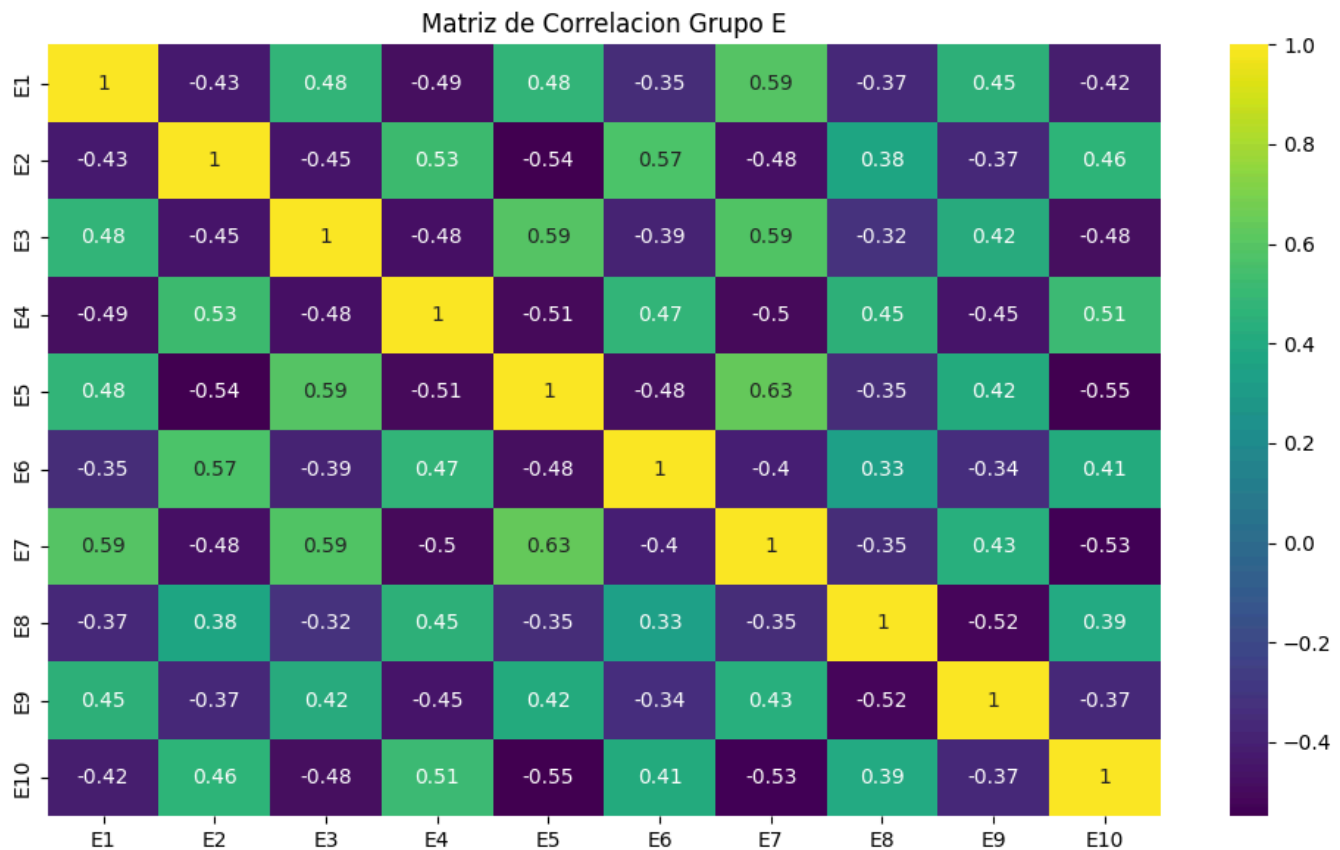
```
# ===== Grafica de Calor =====

# Creamos el Lienzo para La grafica
plt.figure(figsize = (10, 6))

# Sobre La gráfica de calor
sns.heatmap(corSp_E, annot = True, cmap = 'viridis')

# Titulo de La grafica
plt.title('Matriz de Correlacion Grupo E')

plt.tight_layout()
plt.show()
```



Para ver que pregunta tiene mejor correlacion con las demas, haremos un promedio por renglon de la matriz

```
# Promedio por renglo
Pro_Corr = corSp_E.mean(axis = 1)

# Obtenemos la pregunta con mayor promedio
mejor_pregunta = Pro_Corr.idxmax()

# Vemos cual es la mejor pregunta
print(f"La mejor pregunta es: {mejor_pregunta}")
```

La mejor pregunta es: E7

Grupo N

```
# Creamos la matriz de Correlacion con la funcion spearman
corSp_N = tabla_new.iloc[:, 10:20].corr( method = 'spearman')
```

```
# ===== Grafica de Calor =====

# Creamos el lienzo para la grafica
plt.figure(figsize = (10, 6))

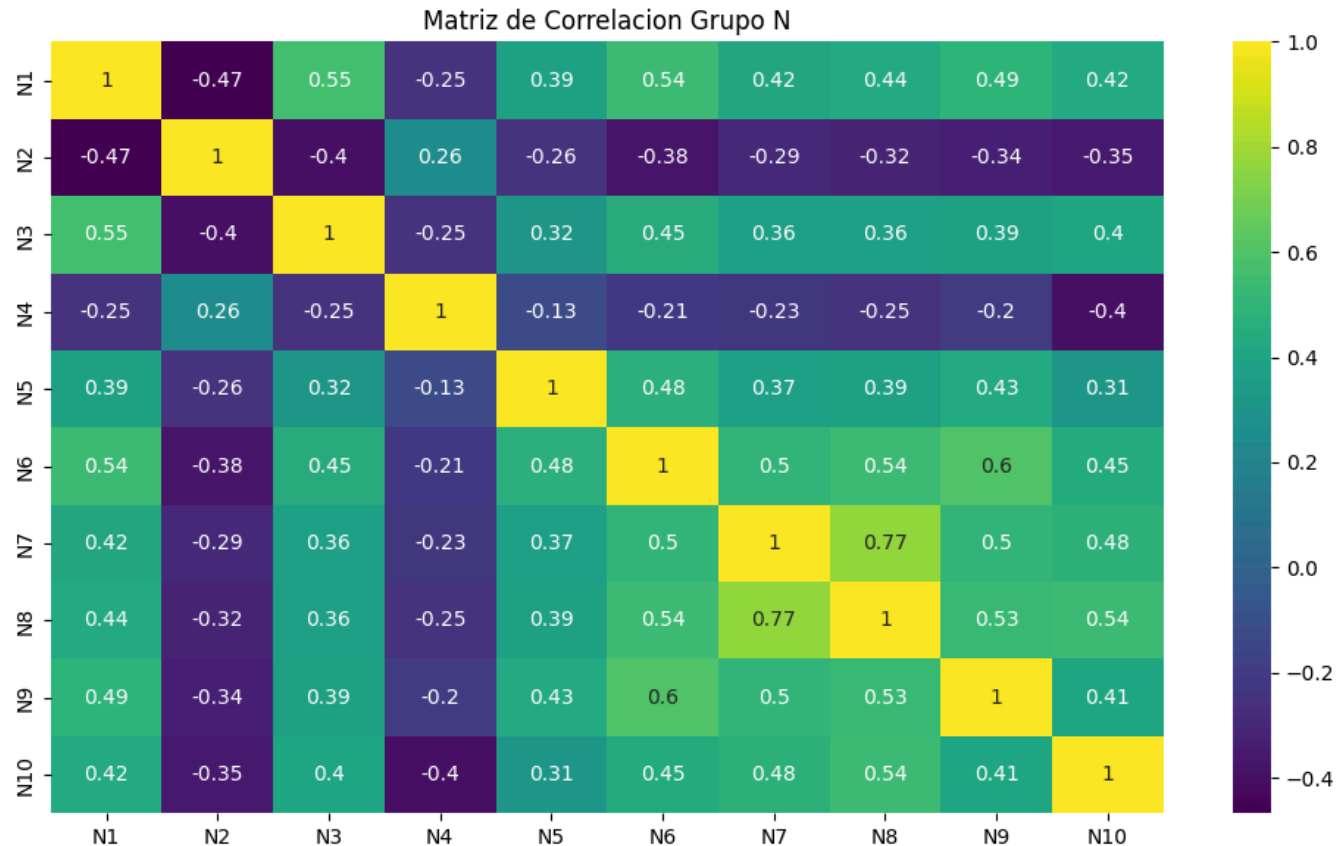
# Sobre la gráfica de calor
sns.heatmap(corSp_N, annot = True, cmap = 'viridis')

# Titulo de la grafica
```

```
plt.title('Matriz de Correlacion Grupo N')
```

```
plt.tight_layout()
```

```
plt.show()
```



Para ver que pregunta tiene mejor correlacion con las demas, haremos un promedio por renglon de la matriz

```
# Promedio por renglo
```

```
Pro_Corr = corSp_N.mean(axis = 1)
```

```
# Obtenemos la pregunta con mayor promedio
```

```
mejor_pregunta = Pro_Corr.idxmax()
```

```
# Vemos cual es la mejor pregunta
```

```
print(f"La mejor pregunta es: {mejor_pregunta}")
```

La mejor pregunta es: N8

Grupo A

```
# Creamos la matriz de Correlacion con la funcion spearman
```

```
corSp_A = tabla_new.iloc[:, 20:30].corr( method = 'spearman')
```

```
# ===== Grafica de Calor =====
```

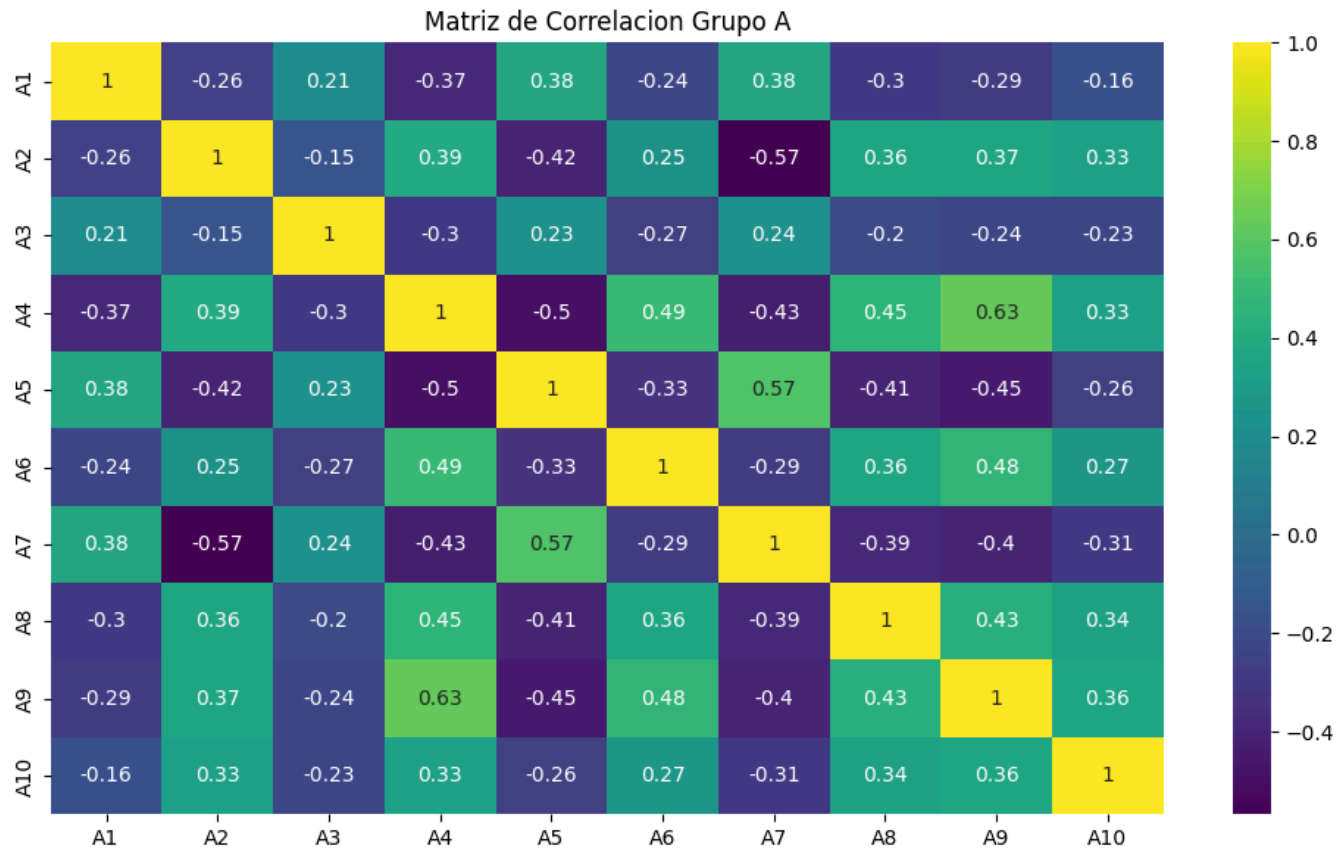
```
# Creamos el lienzo para la grafica
```

```
plt.figure(figsize = (10, 6))
```

```
# Sobre La gráfica de calor
sns.heatmap(corSp_A, annot = True, cmap = 'viridis')

# Titulo de La grafica
plt.title('Matriz de Correlacion Grupo A')

plt.tight_layout()
plt.show()
```



Para ver que pregunta tiene mejor correlacion con las demas, haremos un promedio por renglon de la matriz

```
# Promedio por renglo
Pro_Corr = corSp_A.mean(axis = 1)

# Obtenemos La pregunta con mayor promedio
mejor_pregunta = Pro_Corr.idxmax()

# Vemos cual es La mejor pregunta
print(f"La mejor pregunta es: {mejor_pregunta}")
```

La mejor pregunta es: A9

Grupo C

```
# Creamos La matriz de Correlacion con La funcion spearman
corSp_C = tabla_new.iloc[:, 30:40].corr( method = 'spearman')
```

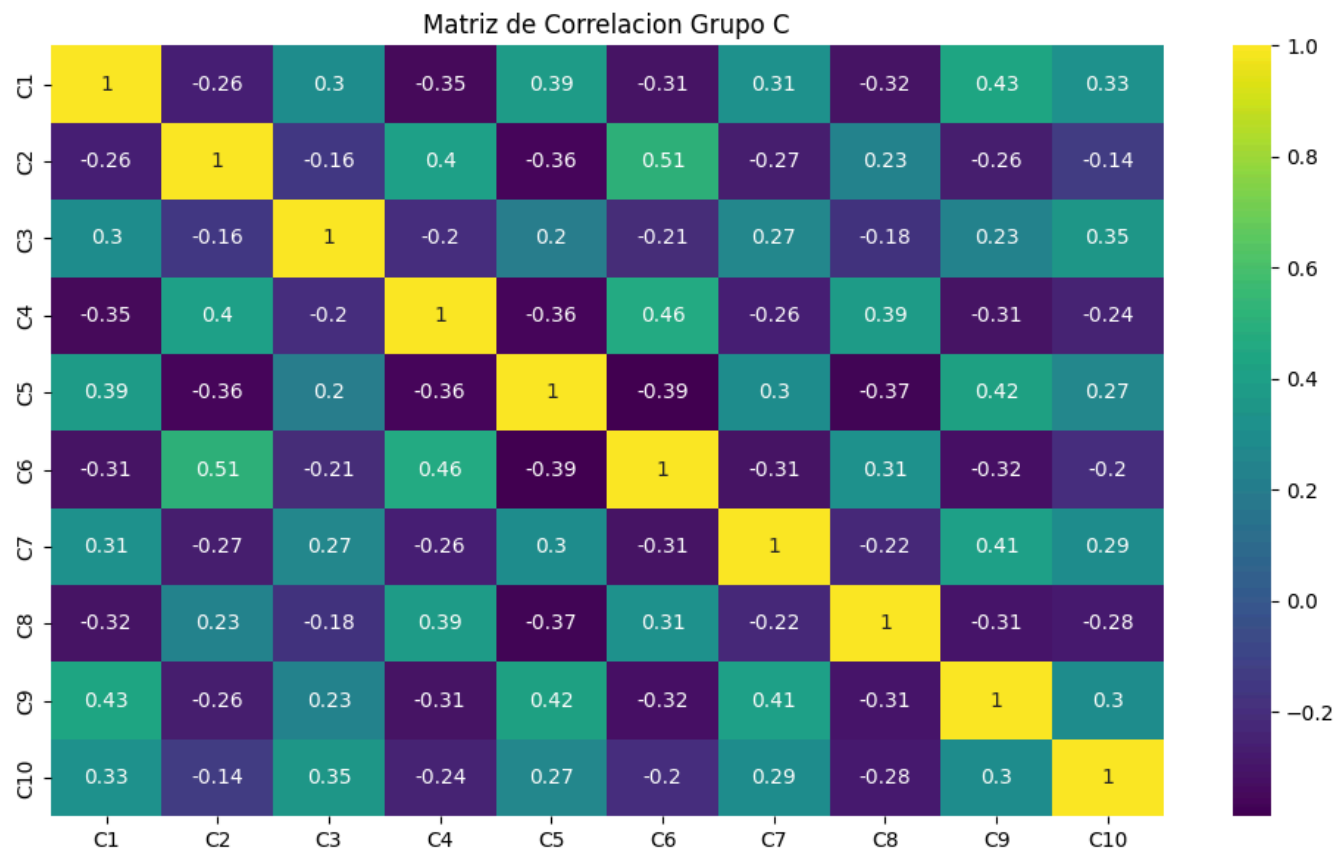
```
# ===== Grafica de Calor =====

# Creamos el lienzo para la grafica
plt.figure(figsize = (10, 6))

# Sobre La gráfica de calor
sns.heatmap(corSp_C, annot = True, cmap = 'viridis')

# Titulo de la grafica
plt.title('Matriz de Correlacion Grupo C')

plt.tight_layout()
plt.show()
```



Para ver que pregunta tiene mejor correlacion con las demas, haremos un promedio por renglon de la matriz

```
# Promedio por renglo
Pro_Corr = corSp_C.mean(axis = 1)

# Obtenemos La pregunta con mayor promedio
mejor_pregunta = Pro_Corr.idxmax()

# Vemos cual es La mejor pregunta
print(f"La mejor pregunta es: {mejor_pregunta}")
```

La mejor pregunta es: C10

Grupo O

```
# Creamos la matriz de Correlacion con la funcion spearman
corSp_0 = tabla_new.iloc[:, 40:50].corr( method = 'spearman')
```

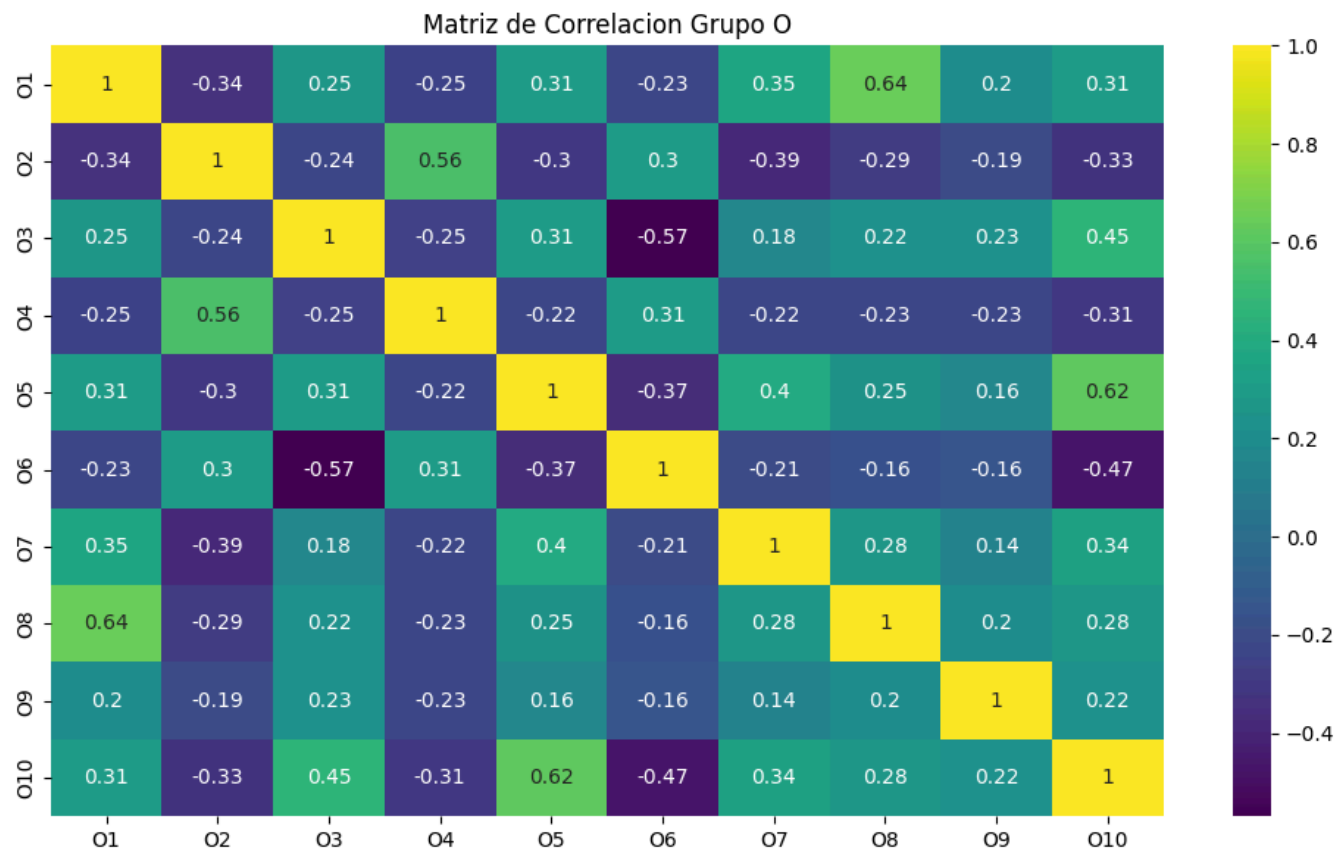
```
# ===== Grafica de Calor =====
```

```
# Creamos el Lienzo para la grafica
plt.figure(figsize = (10, 6))
```

```
# Sobre la gráfica de calor
sns.heatmap(corSp_0, annot = True, cmap = 'viridis')
```

```
# Titulo de la grafica
plt.title('Matriz de Correlacion Grupo O')
```

```
plt.tight_layout()
plt.show()
```



Para ver que pregunta tiene mejor correlacion con las demas, haremos un promedio por renglon de la matriz

```
# Promedio por renglo
```

```
Pro_Corr = corSp_0.mean(axis = 1)
```

```
# Obtenemos la pregunta con mayor promedio
```

```
mejor_pregunta = Pro_Corr.idxmax()
```

```
# Vemos cual es la mejor pregunta
print(f"La mejor pregunta es: {mejor_pregunta}")
```

La mejor pregunta es: 01

Pregunta 3:

****Detallar el proceso de limpieza de datos con los siguientes incisos *(tambien indicando el numero de registros que estamos eliminando)****

Eliminar casos con edades de más de 66. Eliminar casos con códigos de países inválidos (ISO country code). Eliminar cualquier caso con valores faltantes en las variables demográficas.

- **Eliminar todos los casos con valores faltantes (0) en los atributos.**

Solucion:

con los siguientes comando que hara esta parte:

```
# Vemos el # de registros con valores nulos
registro_1 = tabla_new.isnull().any(axis = 1).sum()

print(f"Tenemos {registro_1} registros con valores nulos")
```

Tenemos 2 registros con valores nulos

```
# quitamos los registro con valores nulos
tabla_new_1 = tabla_new.dropna()
```

- **Eliminar casos con edades de más de 66**

Solucion:

con los siguientes comando que hara esta parte:

```
# Vemos el # de registros con edades mayores a los 66 años
registro_2 = (tabla_new_1['age'] > 66).sum()

print(f"Tenemos {registro_2} registros con edades mayores a 66")
```

Tenemos 192 registros con edades mayores a 66

```
# Excluimos los registros con la edad indicada
tabla_new_2 = tabla_new_1[tabla_new_1['age'] <= 66]
```

- ****Eliminar casos con códigos de países inválidos *(ISO country code)****.

Solucion:

Para checar esto hay que investigar por internet si los valores unicos que maneja la columna **Country** son las abreviacionc correctas al pais que desean indican, por lo que si observamos denuevo estos valores...

```
# Valores unicos de la columna "Country"
holis
```

```
array(['US', 'PK', 'RO', 'IN', 'IT', 'FR', 'GB', 'LT', 'CA', 'GR', 'DK',
      'MX', 'SE', 'JO', 'NO', 'IE', 'NZ', 'FI', 'BE', 'HR', 'ZA', 'PL',
      'BR', 'SK', 'SA', 'PH', 'AU', 'JP', 'ID', 'MY', 'KW', 'OM', 'TR',
      'SG', 'IL', 'DE', 'AE', 'BT', 'AR', 'RS', 'GH', 'NL', 'MU', 'KE',
      '(nu', 'AT', 'TZ', 'ES', nan, 'HU', 'RU', 'UG', 'PT', 'HK', 'CO',
      'CN', 'BH', 'ME', 'BZ', 'SI', 'CZ', 'CH', 'LK', 'EG', 'TH', 'BG',
      'AZ', 'PR', 'NP', 'FJ', 'CM', 'BD', 'VN', 'LB', 'PW', 'MA', 'IR',
      'EE', 'JM', 'UA', 'MM', 'A1', 'KR', 'CY', 'JE', 'QA', 'MK', 'VE',
      'IS', 'NG', 'ZW', 'CR', 'CL', 'SY', 'DO', 'AL', 'SV', 'USA', 'TT',
      'EU', 'BM', 'HT', 'TW', 'MZ', 'BW', 'BF', 'TN', 'IM', 'BA', 'KZ',
      'GG', 'SD', 'HN', 'GE', 'KH', 'PG', 'MV', 'A2', 'RW', 'LS', 'PE',
      'BB', 'LA', 'BO', 'VC', 'PA', 'BN', 'AG', 'LV', 'MP', 'EC', 'MT',
      'GP', 'AS', 'AP', 'FO', 'NI', 'GU', 'DZ', 'BS', 'SR', 'GY', 'LY',
      'IQ', 'GT', 'VI', 'KY', 'MN', 'ET', 'AO', 'UY', 'GD', 'MR', 'USS',
      'CV', 'MW', 'ZM', 'PY', 'UZ', 'MXN', 'TC', 'KG'], dtype=object)
```

Ignorando al valor nulo que que esta en la lista (*ya que estamos sobre la base de datos original*) e investigando en internet; tenemos la lista de las abreviaciones que **NO** son correspondiente a un pais (*por el ISO CODE*):

- (nu , PW , A1 , A2 , JE , GG , IM , AP , GP , USS , MXN

Una cosa a aclarar es el hecho de una columna llamada: **NA**) era para determinar si el registro tenia o no datos del pais; fue un error que no considere

Por ende hay que eliminar esta columnas...

```
# Eliminamos esta columnas
```

```
tabla_new_3 = tabla_new_2.drop(columns = ['country_(nu', 'country_PW', 'country_A1', 'country_
```

Pero revisando estas abreviacion; nos topamos con una nueva problematica; es el hecho de que algunos regiones son parte de algun pais y es un problematica que lo podemos resolver con unas cuantas sumas....

Pero primero veamos cuales pertenece al cuales:

- BM , KY , TC pertenece a **los Britanicos**.
- PR , AS , GU , VI pertenece a **los Estados Unidos de America**.

Para este caso, (*como esta separados las columnas*) vamos sumar en la columna del pais con abreviacion incorrecta en los paises que si le pertenece (*esa celdas en cuestion deberia de estar vacias*)

```
# Para el caso de Los britanicos
```

```
tabla_new_3['country_GB'] = tabla_new_3['country_GB'] + tabla_new_3['country_BM'] + tabla_new_
```

```
# Para el caso de Los estados unidos
tabla_new_3['country_EU'] = tabla_new_3['country_EU'] + tabla_new_3['country_PR'] + tabla_new_3['country_TC']

# Luego borramos las columnas anteriores
tabla_new_4 = tabla_new_3.drop(columns = ['country_BM', 'country_KY', 'country_TC', 'country_FJ'])
```

- **Eliminar cualquier caso con valores faltantes en las variables demográficas.**

Solucion:

Cuando convertimos los datos en numeros enteros y con la columnas **Country** realizamos las variables clasificatorias; los valores nulos que tenia fuero excluidos y si sobraba alguno, el primer inciso de encargo de ellos...

Para verificar lo haremos con este codigo.

```
print( f"Tenemos: {tabla_new_4.isnull().any(axis = 1).sum()} datos que son nulos")
```

Tenemos: 0 datos que son nulos

Para concluir este problema; vamos compara las dimensiones que tenia la base de datos original con la que esta limpia e transformada...

```
# Dimensiones de La original
print(f"La base de datos original tenia: {tabla.shape[0]} registros y {tabla.shape[1]} columnas")

# Dimensiones La mas reciente
print(f"La base de datos limpia y transformada tenia: {tabla_new_4.shape[0]} registros y {tabla_new_4.shape[1]} columnas")

# Los registro borrados
print(f"Se borraron: {tabla.shape[0] - tabla_new_4.shape[0]} registros; que son los mismo que se borraron")
```

La base de datos original tenia: 19723 registros y 57 columnas

La base de datos limpia y transformada tenia: 19529 registros y 199 columnas

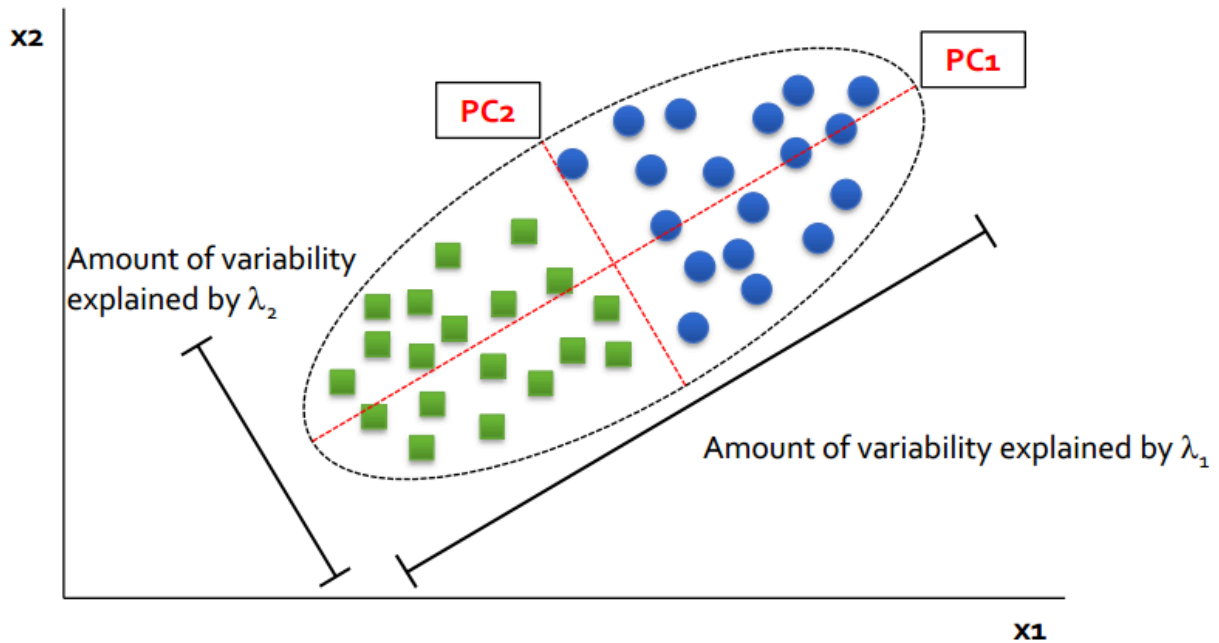
Se borraron: 194 registros; que son los mismo que tenemos conteplados en esta limpieza. 194

Pregunta 4:

Realizar el análisis PCA de los datos, concluye si se pueden reducir el número de variables en componentes.

Solucion:

Recordemos que la funcionalidad del **Analisis de Componentes Principales** es reducir la dimensionalidad de la basede datos N en una dimnesion M (*menor que la original*) con el proposito manejar los datos sin muchas complicaciones por el extensa cantidad de datos que puede ralentizar los procesos de analisis.



Para que la información de la base de datos no se pierda es importante que la **Varianza estadística** se conserve para ello se realiza la búsqueda de:

- **Vectores Propios**
- **Valores Propios**

Con la Matriz de correlación que genera la base de datos (*sintetizando este tema*), para este caso usaremos la base de datos que acabamos de limpiar...

Primero solo vamos a seleccionar todas las partes; excepto las columnas clasificatorias por el país (*ya que no aportan mucha información a los componentes en este estado de V.A.s Clasificatorias*)

```
tabla_pro = tabla_new_4.iloc[:, :56]
```

Segundo, paso a aplicar el modelo de "Componente Principales"

```
# Importamos el modelo PCA
from sklearn.decomposition import PCA

# Guardamos la función y vamos a todo los componentes que puede generar
pca = PCA(n_components = None)

# Ajustamos el modelo
componentes = pca.fit_transform(tabla_pro)
```

Y ya estaría con el modelo hecho; podemos ver algunas cosas a rescatar para concluir mejor el modelo...

- La primera sería: **¿Cuántas variables necesito para representar el 88% o más de los datos?**

En la teoria nos dice que con cubrir el mas de **70%** de **la varianza explicativa del PCA** es mas que suficiente para determinara el numero de componenetes que que necesitamos. Dicha varianza sera representada por lo por los valores propios que si lo graficamos en **una grafica de Codo**

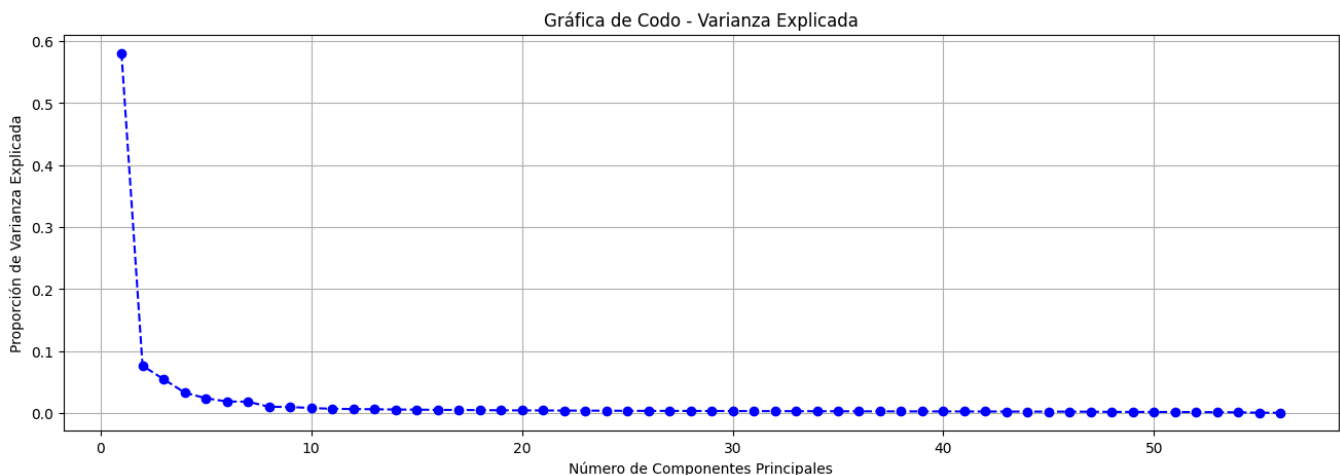
```
# Establecemos el tamaño del lienzo
plt.figure(figsize = (16, 5))

# Los valores a tomar de la grafica: x,y repectivamente
x = range(1, len(pca.explained_variance_ratio_) + 1)
y = pca.explained_variance_ratio_ # Valores propios

# sera una grafica de linea que una Los puntos "y"
plt.plot(x, y, marker='o', linestyle = '--', color = 'b')

# configuramos los elementos de la grafica
plt.title('Gráfica de Codo - Varianza Explicada')
plt.xlabel('Número de Componentes Principales')
plt.ylabel('Proporción de Varianza Explicada')

# Mostramos la grafica
plt.grid(True)
plt.show()
```



Se le llama **Grafica de codo** ya que el punto que "quiebra" y forma el codo no indicaria la cantida de componentes que necesitamos que expresa el 88% de la varianza....

Para este caso diremos que con **25** componentes son mas que suficientes; ya que si vemos la propocion que ofrece cada uno de esto componentes....

```
# ===== Sumamos Toda La Varianza =====
# aqui estara la suma y empieza con 0
Sumvp1 = 0

# el "for" hara la suma totalmente
for i in range(0, len(y)):
    Sumvp1 += y[i]
```

```
# ===== Vemos La proporcion de varianza que ofrece =====

# con esto podemos calcular las proporciones de cada componente.
contador = 0

# este variable suma la varianza hasta el caso presente
PT = 0

# el bucle " no ayuda a calcular las proporcion"
while contador <= 24:
    P1 = (y[contador] / Sumvp1) * 100
    print( f"La Proporcion del PC{contador + 1} es: {P1: .3f}%" )
    contador += 1
    PT += P1

print( f"Representa una variabilidad del {PT: .3f}%" )
```

```
La Proporcion del PC1 es: 58.063%
La Proporcion del PC2 es: 7.659%
La Proporcion del PC3 es: 5.491%
La Proporcion del PC4 es: 3.332%
La Proporcion del PC5 es: 2.422%
La Proporcion del PC6 es: 1.899%
La Proporcion del PC7 es: 1.868%
La Proporcion del PC8 es: 1.052%
La Proporcion del PC9 es: 1.009%
La Proporcion del PC10 es: 0.846%
La Proporcion del PC11 es: 0.718%
La Proporcion del PC12 es: 0.678%
La Proporcion del PC13 es: 0.648%
La Proporcion del PC14 es: 0.603%
La Proporcion del PC15 es: 0.587%
La Proporcion del PC16 es: 0.544%
La Proporcion del PC17 es: 0.531%
La Proporcion del PC18 es: 0.507%
La Proporcion del PC19 es: 0.477%
La Proporcion del PC20 es: 0.459%
La Proporcion del PC21 es: 0.457%
La Proporcion del PC22 es: 0.432%
La Proporcion del PC23 es: 0.423%
La Proporcion del PC24 es: 0.416%
La Proporcion del PC25 es: 0.392%
Representa una variabilidad del 91.510%
```

- La Segunda seria: **¿Cuáles son las variables representativas para el análisis posterior?**

Para eso vamos a formalizar estos componentes en un tabla y el nombre de las columnas ya que:

- Cada fila sera el componente
- cada Columna sera la parte de la V.A, que representa *(los componente tiene cachos de varianza de cada V.A.s que tiene la base)*

```
PCA_list = pd.DataFrame(pca.components_, columns = tabla_pro.columns, index = [f'PC{i+1}' for
```

```
# Visualizamos lo datos pero de los 22 componentes que seleccionamos
```

```
PCA_list_selec = PCA_list.iloc[:25,:]
```

```
PCA_list_selec
```

	E1	E2	E3	E4	E5	E6	E7	E8	E9
PC1	0.003923	-0.006726	0.013554	-0.009954	0.016331	-0.014508	0.012129	-0.002557	-0.000514
PC2	0.010421	0.002377	0.023538	0.000657	0.016130	0.022459	0.005578	-0.004273	0.009136
PC3	0.202120	-0.209435	0.259029	-0.221639	0.246571	-0.200945	0.276538	-0.158055	0.211812
PC4	0.162426	-0.213194	0.094310	-0.132260	0.185478	-0.147690	0.200015	-0.151805	0.176298
PC5	-0.081459	0.061845	-0.001882	0.086383	-0.012172	0.029390	-0.057391	0.135928	-0.133287
PC6	0.127670	-0.103315	0.034961	-0.122885	0.092148	-0.041098	0.118774	-0.145244	0.170343
PC7	-0.038355	0.049864	-0.084279	0.065658	-0.024909	-0.073729	-0.069847	0.027969	0.036148
PC8	-0.058140	-0.131774	-0.067085	-0.100403	-0.017339	-0.168622	-0.032093	0.015062	-0.116470
PC9	0.099611	0.142276	0.081657	0.135039	0.015373	0.173679	0.070779	0.103959	0.052962
PC10	0.048675	0.142645	-0.023631	0.018666	-0.080584	0.098993	-0.031782	-0.139836	0.157626
PC11	0.107399	0.085115	0.112494	0.096742	0.139867	0.100501	0.280834	0.306097	-0.217792
PC12	-0.013432	0.219953	-0.048588	-0.041462	-0.165805	0.174533	-0.154553	-0.449074	0.528071
PC13	-0.003613	-0.042779	-0.014519	-0.032960	-0.085532	-0.039152	-0.098452	0.008392	0.021486
PC14	0.033573	0.053398	-0.015111	-0.005356	0.046498	0.090716	0.098612	0.006951	-0.032073
PC15	0.151765	0.129020	0.011047	0.093760	-0.012511	0.087470	0.095421	0.008891	0.217533
PC16	-0.213246	-0.263084	-0.052334	0.031663	-0.036490	-0.326707	-0.331821	-0.139771	-0.012727
PC17	0.006890	-0.146681	-0.097527	-0.024462	-0.015804	-0.111563	-0.110023	-0.043891	-0.084605
PC18	0.092609	0.197020	-0.078647	0.073585	-0.041551	0.192556	0.165854	-0.103319	-0.065049
PC19	-0.112087	0.088129	0.077058	0.054869	0.016701	0.116980	-0.004976	-0.108377	-0.102473
PC20	-0.207145	-0.126259	-0.077406	0.074454	0.085208	-0.072381	-0.008919	-0.089049	-0.088625
PC21	0.023206	-0.199114	-0.175733	-0.013928	0.042528	-0.053546	-0.053502	0.043396	-0.009856
PC22	-0.056758	0.054182	0.043659	-0.020822	-0.001972	0.020914	-0.006882	0.112527	0.127261
PC23	0.098326	-0.065131	0.090292	-0.114846	-0.028967	0.058989	0.015357	0.067836	-0.007159
PC24	0.298877	-0.166420	-0.081225	-0.033031	-0.108409	-0.207199	0.119614	0.104504	0.081689
PC25	-0.373456	-0.006653	0.141259	0.142813	0.200675	-0.047537	0.019350	0.133068	0.248795

25 rows × 56 columns



Para detectar que V.A.s se representa a los componentes escogemos a la de mayor valor y lo podemos ver en el siguiente código...

```
for i in range(PCA_list_selec.shape[0]):  
    print(f"\nVariables más influyentes en PC{i+1}:")  
    print(PCA_list_selec.iloc[i].abs().sort_values(ascending = False).head(1))
```

Variables más influyentes en PC1:

age 0.995037

Name: PC1, dtype: float64

Variables más influyentes en PC2:

race 0.984267

Name: PC2, dtype: float64

Variables más influyentes en PC3:

E7 0.276538

Name: PC3, dtype: float64

Variables más influyentes en PC4:

N8 0.279917

Name: PC4, dtype: float64

Variables más influyentes en PC5:

C9 0.278508

Name: PC5, dtype: float64

Variables más influyentes en PC6:

A1 0.269817

Name: PC6, dtype: float64

Variables más influyentes en PC7:

O8 0.39733

Name: PC7, dtype: float64

Variables más influyentes en PC8:

source 0.765593

Name: PC8, dtype: float64

Variables más influyentes en PC9:

source 0.530863

Name: PC9, dtype: float64

Variables más influyentes en PC10:

source 0.285796

Name: PC10, dtype: float64

Variables más influyentes en PC11:

E8 0.306097

Name: PC11, dtype: float64

Variables más influyentes en PC12:

E9 0.528071

Name: PC12, dtype: float64

Variables más influyentes en PC13:

N4 0.438039

Name: PC13, dtype: float64

Variables más influyentes en PC14:

A1 0.354605

Name: PC14, dtype: float64

Variables más influyentes en PC15:

A3 0.345249

Name: PC15, dtype: float64

Variables más influyentes en PC16:

A1 0.487319

Name: PC16, dtype: float64

Variables más influyentes en PC17:

O4 0.505837

Name: PC17, dtype: float64

Variables más influyentes en PC18:

N4 0.388232

Name: PC18, dtype: float64

Variables más influyentes en PC19:

N5 0.439013

Name: PC19, dtype: float64

Variables más influyentes en PC20:

C7 0.447198

Name: PC20, dtype: float64

Variables más influyentes en PC21:

C5 0.357993

Name: PC21, dtype: float64

Variables más influyentes en PC22:

C6 0.601729

Name: PC22, dtype: float64

Variables más influyentes en PC23:

C9 0.353862

Name: PC23, dtype: float64

Variables más influyentes en PC24:

E10 0.599928

Name: PC24, dtype: float64

Variables más influyentes en PC25:

O9 0.403455

Name: PC25, dtype: float64

Realiza el análisis de clustering para 2, 3, y 4 clústeres, ¿qué clúster tiene más datos? ¿Cuántos clústeres recomiendas generar y por qué? Es decir, k=?

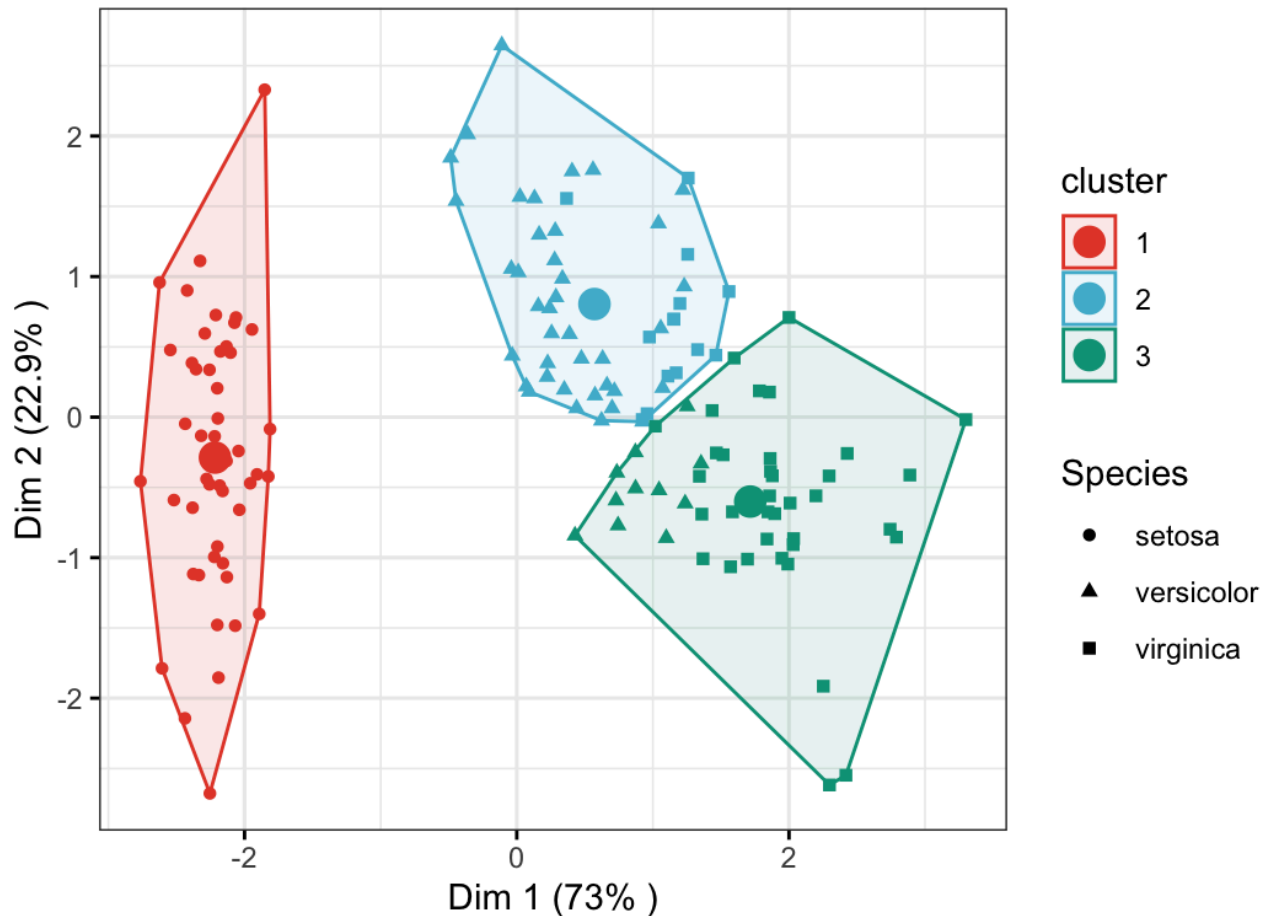
Realiza un análisis descriptivo de los datos generados en 2 o más clústeres.

Pregunta 5:

Realiza el análisis de clustering para 2, 3, y 4 clústeres

Solucion:

Los modelos **Clustering** son modelos del tipo **Aprendizaje no supervisado** (ya que no conocemos las etiquetas de cada registro) y nos ayuda a determinar una cierta cantidad de grupos que agrupan dicho registro y así podemos clasificar los datos...



Lo mas comun para decir que un registro se parece a otra es por medio de las distancia. Para esta ocasion estaremos usando **las distancia Manhanta...**

Distancia de Manhattan: Se realiza una suma de dichas diferencias por entrada.

$$d(x_i, x_j) = \sum_{p=1}^n |x_{ip} - x_{jp}|$$

Ya que la mayoría de V.A.s esta en formato nominal y por ello los resultados sera mas sencillo de calcular. (sera con toda la tabla limpia)...

Pasando a responder algunas de las preguntas: **¿Cuántos clústeres recomiendas generar y por qué?**
Es decir, k=?.

Existe algo que se llama **Dendrograma** sera una grafica que estar uniendo los datos con una linea de acuerdo a la cercania que tiene los datos por jerarquias y se agrupara por **ramas**.

El numero de remas finales sera el numero de Clusterings que debemos formar para este modelo.

Para este caso vamos **estandarizar** que los procesos sean mas ligeros...

```
# importamos la funcion de estandarizacion
from sklearn.preprocessing import StandardScaler

# Escalar los datos
scaler = StandardScaler()
tabla_escalada = scaler.fit_transform(tabla_new_4)
```

Pasando a la grafica:

```
#===== Preparamos los datos para la grafica =====

# Importamos 2 funcion: dendrogram que sera la grafica y la funcion linkage que nos ayuda en l
from scipy.cluster.hierarchy import dendrogram, linkage

# Tambien importamos la distancias
from scipy.spatial.distance import pdist

# Calcular las distancias usando la métrica de Manhattan
distancias = pdist(tabla_escalada, metric = 'cityblock')

# Calcular el linkage usando como parametro el promedio
Z = linkage(distancias, method = 'average')
```

```
#===== Grafica del Dendrograma =====

# Establecemos el tamaño del Lienzo
plt.figure(figsize = (25, 8))

# Indicamos que datos a graficar
dendrogram(Z)

# Configuramos los elementos de la grafica
plt.title("Dendrograma con distancia Manhattan")
plt.xlabel("Observaciones")
plt.ylabel("Distancia (Manhattan)")

# Visualizamos la tabla
plt.show()
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipython-input-330-521548095.py in <cell line: 0>()
      5
      6 # Indicamos que datos a graficar
----> 7 dendrogram(Z)
      8
      9 # Configuramos lo elementos de la grafica

/usr/local/lib/python3.11/dist-packages/scipy/cluster/hierarchy.py in dendrogram(Z, p, truncate_mode, color_threshold, get_leaves, orientation, labels, count_sort, distance_sort, show_leaf_counts, no_plot, no_labels, leaf_font_size, leaf_rotation, leaf_label_func, show_contracted, link_color_func, ax, above_threshold_color)
    3362     if not no_plot:
    3363         mh = xp.max(Z[:, 2])
-> 3364         _plot_dendrogram(icoord_list, dcoord_list, ivl, p, n, mh, orientation,
    3365                         no_labels, color_list,
    3366                         leaf_font_size=leaf_font_size,

/usr/local/lib/python3.11/dist-packages/scipy/cluster/hierarchy.py in _plot_dendrogram(icoords, dcoords, ivl, p, n, mh, orientation, no_labels, color_list, leaf_font_size, leaf_rotation, contraction_marks, ax, above_threshold_color)
    2830         ax.set_xticklabels([])
    2831     else:
-> 2832         ax.set_xticks(iv_ticks)
    2833
    2834         if orientation == 'top':

/usr/local/lib/python3.11/dist-packages/matplotlib/axes/_base.py in wrapper(self, *args, **kwargs)
     72
     73     def wrapper(self, *args, **kwargs):
---> 74         return get_method(self)(*args, **kwargs)
     75
     76     wrapper.__module__ = owner.__module__

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in set_ticks(self, ticks, labels, minor, **kwargs)
    2227         "other than 'minor' modify the text labels and can only be used if "
    2228         "'labels' are passed as well.")
-> 2229     result = self._set_tick_locations(ticks, minor=minor)
    2230     if labels is not None:
    2231         self.set_ticklabels(labels, minor=minor, **kwargs)

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in _set_tick_locations(self, ticks, minor)
    2179     else:
    2180         self.set_major_locator(locator)
-> 2181         return self.get_major_ticks(len(ticks))
    2182
    2183     def set_ticks(self, ticks, labels=None, *, minor=False, **kwargs):

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in get_major_ticks(self, numticks)
    1673     while len(self.majorTicks) < numticks:
    1674         # Update the new tick label properties from the old.
-> 1675         tick = self._get_tick(major=True)

```

```

1676         self.majorTicks.append(tick)
1677         self._copy_tick_props(self.majorTicks[0], tick)

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in _get_tick(self, major)
1601         "_tick_class or reimplement _get_tick()")
1602         tick_kw = self._major_tick_kw if major else self._minor_tick_kw
-> 1603         return self._tick_class(self.axes, 0, major=major, **tick_kw)
1604
1605     def _get_tick_label_size(self, axis_name):

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in __init__(self, *args, **kwargs)
366
367     def __init__(self, *args, **kwargs):
--> 368         super().__init__(*args, **kwargs)
369         # x in data coords, y in axes coords
370         ax = self.axes

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in __init__(self, axes, loc, size,
width, color, tickdir, pad, labelsize, labelcolor, labelfontfamily, zorder, gridOn, tick1On,
tick2On, label1On, label2On, major, labelrotation, grid_color, grid_linestyle, grid_linewidth
h, grid_alpha, **kwargs)
152         grid_kw = {k[5:]: v for k, v in kwargs.items()}
153
--> 154         self.tick1line = mlines.Line2D(
155             [], [],
156             color=color, linestyle="none", zorder=zorder, visible=tick1On,

/usr/local/lib/python3.11/dist-packages/matplotlib/lines.py in __init__(self, xdata, ydata, l
inewidth, linestyle, color, gapcolor, marker, markersize, markeredgewidth, markeredgewidth, m
arkerfacecolor, markerfacecoloralt, fillstyle, antialiased, dash_capstyle, solid_capstyle, da
sh_joinstyle, solid_joinstyle, pickradius, drawstyle, markevery, **kwargs)
378         marker = 'none' # Default.
379         if not isinstance(marker, MarkerStyle):
--> 380             self._marker = MarkerStyle(marker, fillstyle)
381         else:
382             self._marker = marker

/usr/local/lib/python3.11/dist-packages/matplotlib/markers.py in __init__(self, marker, fills
tyle, transform, capstyle, joinstyle)
246         self._user_joinstyle = JoinStyle(joinstyle) if joinstyle is not None else Non
e
247         self._set_fillstyle(fillstyle)
--> 248         self._set_marker(marker)
249
250     def _recache(self):

/usr/local/lib/python3.11/dist-packages/matplotlib/markers.py in _set_marker(self, marker)
332         if not isinstance(marker, MarkerStyle):
333             self._marker = marker
--> 334             self._recache()
335
336     def get_path(self):

/usr/local/lib/python3.11/dist-packages/matplotlib/markers.py in _recache(self)
252         return
253         self._path = _empty_path

```

```

--> 254         self._transform = IdentityTransform()
      255         self._alt_path = None
      256         self._alt_transform = None

/usr/local/lib/python3.11/dist-packages/matplotlib/transforms.py in __init__(self, *args, **k
wargs)
      1768
      1769     def __init__(self, *args, **kwargs):
-> 1770         super().__init__(*args, **kwargs)
      1771         self._inverted = None
      1772

/usr/local/lib/python3.11/dist-packages/matplotlib/transforms.py in __init__(self, shorthand_
name)
      108         """
      109
-> 110     def __init__(self, shorthand_name=None):
      111         """
      112         Parameters

```

KeyboardInterrupt:

Error in callback <function _draw_all_if_interactive at 0x7a90fc1a3ce0> (for post_execute):

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
/usr/local/lib/python3.11/dist-packages/matplotlib/pyplot.py in _draw_all_if_interactive()
    277 def _draw_all_if_interactive() -> None:
    278     if matplotlib.is_interactive():
--> 279         draw_all()
    280
    281

/usr/local/lib/python3.11/dist-packages/matplotlib/_pylab_helpers.py in draw_all(cls, force)
    129     for manager in cls.get_all_fig_managers():
    130         if force or manager.canvas.figure.stale:
--> 131             manager.canvas.draw_idle()
    132
    133

/usr/local/lib/python3.11/dist-packages/matplotlib/backend_bases.py in draw_idle(self, *args,
**kwargs)
    1889         if not self._is_idle_drawing:
    1890             with self._idle_draw_cntx():
-> 1891                 self.draw(*args, **kwargs)
    1892
    1893     @property

/usr/local/lib/python3.11/dist-packages/matplotlib/backends/backend_agg.py in draw(self)
    380         with (self.toolbar._wait_cursor_for_draw_cm() if self.toolbar
    381             else nullcontext()):
--> 382             self.figure.draw(self.renderer)
    383             # A GUI class may be need to update a window using this draw, so
    384             # don't forget to call the superclass.

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in draw_wrapper(artist, rendere
r, *args, **kwargs)
    92     @wraps(draw)
    93     def draw_wrapper(artist, renderer, *args, **kwargs):
---> 94         result = draw(artist, renderer, *args, **kwargs)
    95         if renderer._rasterizing:
    96             renderer.stop_rasterizing()

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in draw_wrapper(artist, rendere
r)
    69         renderer.start_filter()
    70
---> 71         return draw(artist, renderer)
    72     finally:
    73         if artist.get_agg_filter() is not None:

/usr/local/lib/python3.11/dist-packages/matplotlib/figure.py in draw(self, renderer)
    3255
    3256         self.patch.draw(renderer)
-> 3257         mimage._draw_list_compositing_images(
    3258             renderer, self, artists, self.suppressComposite)
    3259

/usr/local/lib/python3.11/dist-packages/matplotlib/image.py in _draw_list_compositing_images
(renderer, parent, artists, suppress_composite)

```

```

132     if not_composite or not has_images:
133         for a in artists:
--> 134             a.draw(renderer)
135     else:
136         # Composite any adjacent images together

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in draw_wrapper(artist, renderer)
    69         renderer.start_filter()
    70
---> 71         return draw(artist, renderer)
    72     finally:
    73         if artist.get_agg_filter() is not None:

/usr/local/lib/python3.11/dist-packages/matplotlib/axes/_base.py in draw(self, renderer)
   3179         _draw_rasterized(self.get_figure(root=True), artists_rasterized, renderer)
   3180
-> 3181         mimage._draw_list_compositing_images(
   3182             renderer, self, artists, self.get_figure(root=True).suppressComposite)
   3183

/usr/local/lib/python3.11/dist-packages/matplotlib/image.py in _draw_list_compositing_images(renderer, parent, artists, suppress_composite)
    132     if not_composite or not has_images:
    133         for a in artists:
--> 134             a.draw(renderer)
    135     else:
    136         # Composite any adjacent images together

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in draw_wrapper(artist, renderer)
    69         renderer.start_filter()
    70
---> 71         return draw(artist, renderer)
    72     finally:
    73         if artist.get_agg_filter() is not None:

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in draw(self, renderer)
   1413         renderer.open_group(__name__, gid=self.get_gid())
   1414
-> 1415         ticks_to_draw = self._update_ticks()
   1416         tlb1, tlb2 = self._get_ticklabel_bboxes(ticks_to_draw, renderer)
   1417

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in _update_ticks(self)
   1292         major_locs = self.get_majorticklocs()
   1293         major_labels = self.major.formatter.format_ticks(major_locs)
-> 1294         major_ticks = self.get_major_ticks(len(major_locs))
   1295         for tick, loc, label in zip(major_ticks, major_locs, major_labels):
   1296             tick.update_position(loc)

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in get_major_ticks(self, numticks)
   1673         while len(self.majorTicks) < numticks:
   1674             # Update the new tick label properties from the old.
-> 1675             tick = self._get_tick(major=True)

```

```

1676         self.majorTicks.append(tick)
1677         self._copy_tick_props(self.majorTicks[0], tick)

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in _get_tick(self, major)
1601         "_tick_class or reimplement _get_tick()")
1602         tick_kw = self._major_tick_kw if major else self._minor_tick_kw
-> 1603         return self._tick_class(self.axes, 0, major=major, **tick_kw)
1604
1605     def _get_tick_label_size(self, axis_name):

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in __init__(self, *args, **kwargs)
366
367     def __init__(self, *args, **kwargs):
--> 368         super().__init__(*args, **kwargs)
369         # x in data coords, y in axes coords
370         ax = self.axes

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in __init__(self, axes, loc, size,
width, color, tickdir, pad, labelsize, labelcolor, labelfontfamily, zorder, gridOn, tick1On,
tick2On, label1On, label2On, major, labelrotation, grid_color, grid_linestyle, grid_linewidth
h, grid_alpha, **kwargs)
152         grid_kw = {k[5:]: v for k, v in kwargs.items()}
153
--> 154         self.tick1line = mlines.Line2D(
155             [], [],
156             color=color, linestyle="none", zorder=zorder, visible=tick1On,

/usr/local/lib/python3.11/dist-packages/matplotlib/lines.py in __init__(self, xdata, ydata, l
inewidth, linestyle, color, gapcolor, marker, markersize, markeredgewidth, markeredgewidth, m
arkerfacecolor, markerfacecoloralt, fillstyle, antialiased, dash_capstyle, solid_capstyle, da
sh_joinstyle, solid_joinstyle, pickradius, drawstyle, markevery, **kwargs)
323
324         # Convert sequences to NumPy arrays.
--> 325         if not np.iterable(xdata):
326             raise RuntimeError('xdata must be a sequence')
327         if not np.iterable(ydata):

/usr/local/lib/python3.11/dist-packages/numpy/lib/_function_base_impl.py in iterable(y)
341
342
--> 343 @set_module('numpy')
344 def iterable(y):
345     """

```

KeyboardInterrupt:

Error in callback <function flush_figures at 0x7a90fc1a1800> (for post_execute):

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
/usr/local/lib/python3.11/dist-packages/matplotlib_inline/backend_inline.py in flush_figures
()
    124         # ignore the tracking, just draw and close all figures
    125         try:
--> 126             return show(True)
    127         except Exception as e:
    128             # safely show traceback if in IPython, else raise

/usr/local/lib/python3.11/dist-packages/matplotlib_inline/backend_inline.py in show(close, block)
    88         try:
    89             for figure_manager in Gcf.get_all_fig_managers():
---> 90                 display(
    91                     figure_manager.canvas.figure,
    92                     metadata=_fetch_figure_metadata(figure_manager.canvas.figure)

/usr/local/lib/python3.11/dist-packages/IPython/core/display.py in display(include, exclude, metadata, transient, display_id, *objs, **kwargs)
    318         publish_display_data(data=obj, metadata=metadata, **kwargs)
    319     else:
--> 320         format_dict, md_dict = format(obj, include=include, exclude=exclude)
    321         if not format_dict:
    322             # nothing to display (e.g. _ipython_display_ took over)

/usr/local/lib/python3.11/dist-packages/IPython/core/formatters.py in format(self, obj, include, exclude)
    178         md = None
    179         try:
--> 180             data = formatter(obj)
    181         except:
    182             # FIXME: log the exception

<decorator-gen-2> in __call__(self, obj)

/usr/local/lib/python3.11/dist-packages/IPython/core/formatters.py in catch_format_error(method, self, *args, **kwargs)
    222         """show traceback on failed format call"""
    223         try:
--> 224             r = method(self, *args, **kwargs)
    225         except NotImplementedError:
    226             # don't warn on NotImplementedError

/usr/local/lib/python3.11/dist-packages/IPython/core/formatters.py in __call__(self, obj)
    339             pass
    340         else:
--> 341             return printer(obj)
    342             # Finally look for special method names
    343             method = get_real_method(obj, self.print_method)

/usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py in print_figure(fig, fmt, bbox_inches, base64, **kwargs)
    149         FigureCanvasBase(fig)
    150
--> 151         fig.canvas.print_figure(bytes_io, **kw)

```

```

152     data = bytes_io.getvalue()
153     if fmt == 'svg':

/usr/local/lib/python3.11/dist-packages/matplotlib/backend_bases.py in print_figure(self, filename, dpi, facecolor, edgecolor, orientation, format, bbox_inches, pad_inches, bbox_extra_artists, backend, **kwargs)
    2153         # so that we can inject the orientation
    2154         with getattr(renderer, "_draw_disabled", nullcontext)():
-> 2155             self.figure.draw(renderer)
    2156         if bbox_inches:
    2157             if bbox_inches == "tight":

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in draw_wrapper(artist, renderer, *args, **kwargs)
    92     @wraps(draw)
    93     def draw_wrapper(artist, renderer, *args, **kwargs):
---> 94         result = draw(artist, renderer, *args, **kwargs)
    95         if renderer._rasterizing:
    96             renderer.stop_rasterizing()

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in draw_wrapper(artist, renderer)
    69         renderer.start_filter()
    70
---> 71         return draw(artist, renderer)
    72     finally:
    73         if artist.get_agg_filter() is not None:

/usr/local/lib/python3.11/dist-packages/matplotlib/figure.py in draw(self, renderer)
   3255
   3256         self.patch.draw(renderer)
-> 3257         mimage._draw_list_compositing_images(
   3258             renderer, self, artists, self.suppressComposite)
   3259

/usr/local/lib/python3.11/dist-packages/matplotlib/image.py in _draw_list_compositing_images(renderer, parent, artists, suppress_composite)
   132     if not_composite or not has_images:
   133         for a in artists:
--> 134             a.draw(renderer)
   135     else:
   136         # Composite any adjacent images together

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in draw_wrapper(artist, renderer)
    69         renderer.start_filter()
    70
---> 71         return draw(artist, renderer)
    72     finally:
    73         if artist.get_agg_filter() is not None:

/usr/local/lib/python3.11/dist-packages/matplotlib/axes/_base.py in draw(self, renderer)
   3179         _draw_rasterized(self.get_figure(root=True), artists_rasterized, renderer)
   3180
-> 3181         mimage._draw_list_compositing_images(

```

```

3182         renderer, self, artists, self.get_figure(root=True).suppressComposite)
3183

/usr/local/lib/python3.11/dist-packages/matplotlib/image.py in _draw_list_compositing_images
(renderer, parent, artists, suppress_composite)
    132     if not_composite or not has_images:
    133         for a in artists:
--> 134             a.draw(renderer)
    135     else:
    136         # Composite any adjacent images together

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in draw_wrapper(artist, rendere
r)
    69         renderer.start_filter()
    70
---> 71         return draw(artist, renderer)
    72     finally:
    73         if artist.get_agg_filter() is not None:

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in draw(self, renderer)
   1413         renderer.open_group(__name__, gid=self.get_gid())
   1414
-> 1415         ticks_to_draw = self._update_ticks()
   1416         tlb1, tlb2 = self._get_ticklabel_bboxes(ticks_to_draw, renderer)
   1417

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in _update_ticks(self)
   1292         major_locs = self.get_majorticklocs()
   1293         major_labels = self.major.formatter.format_ticks(major_locs)
-> 1294         major_ticks = self.get_major_ticks(len(major_locs))
   1295         for tick, loc, label in zip(major_ticks, major_locs, major_labels):
   1296             tick.update_position(loc)

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in get_major_ticks(self, numticks)
   1673         while len(self.majorTicks) < numticks:
   1674             # Update the new tick label properties from the old.
-> 1675             tick = self._get_tick(major=True)
   1676             self.majorTicks.append(tick)
   1677             self._copy_tick_props(self.majorTicks[0], tick)

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in _get_tick(self, major)
   1601         "_tick_class or reimplement _get_tick()")
   1602         tick_kw = self._major_tick_kw if major else self._minor_tick_kw
-> 1603         return self._tick_class(self.axes, 0, major=major, **tick_kw)
   1604
   1605     def _get_tick_label_size(self, axis_name):

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in __init__(self, *args, **kwargs)
   366
   367     def __init__(self, *args, **kwargs):
--> 368         super().__init__(*args, **kwargs)
   369         # x in data coords, y in axes coords
   370         ax = self.axes

/usr/local/lib/python3.11/dist-packages/matplotlib/axis.py in __init__(self, axes, loc, size,
width, color, tickdir, pad, labelsize, labelcolor, labelfontfamily, zorder, gridOn, tick10n,

```

```

tick20n, label10n, label20n, major, labelrotation, grid_color, grid_linestyle, grid_linewidth,
h, grid_alpha, **kwargs)
    152         grid_kw = {k[5:]: v for k, v in kwargs.items()}
    153
--> 154         self.tick1line = mlines.Line2D(
    155             [], [],
    156             color=color, linestyle="none", zorder=zorder, visible=tick10n,

/usr/local/lib/python3.11/dist-packages/matplotlib/lines.py in __init__(self, xdata, ydata, linewidth,
linestyle, color, gapcolor, marker, markersize, markeredgewidth, markeredgewidth, markeredgewidth, markeredgewidth, m
arkerfacecolor, markerfacecoloralt, fillstyle, antialiased, dash_capstyle, solid_capstyle, dash_joistyle, solid_joistyle, pickradius, drawstyle, markevery, **kwargs)
    405         # update kwargs before updating data to give the caller a
    406         # chance to init axes (and hence unit support)
--> 407         self._internal_update(kwargs)
    408         self.pickradius = pickradius
    409         self.ind_offset = 0

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in _internal_update(self, kwargs)
    1231         The lack of prenormalization is to maintain backcompatibility.
    1232         """
-> 1233         return self._update_props(
    1234             kwargs, "{cls.__name__}.set() got an unexpected keyword argument "
    1235             "{prop_name!r}")

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in _update_props(self, props, errfmt)
    1207             errfmt.format(cls=type(self), prop_name=k),
    1208             name=k)
-> 1209         ret.append(func(v))
    1210         if ret:
    1211             self.pchanged()

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in set_zorder(self, level)
    1135         if level != self.zorder:
    1136             self.zorder = level
-> 1137             self.pchanged()
    1138             self.stale = True
    1139

/usr/local/lib/python3.11/dist-packages/matplotlib/artist.py in pchanged(self)
    423         remove_callback
    424         """
--> 425         self._callbacks.process("pchanged")
    426
    427         def is_transform_set(self):

/usr/local/lib/python3.11/dist-packages/matplotlib/cbook.py in process(self, s, *args, **kwargs)
    354         """
    355         if self._signals is not None:
--> 356             _api.check_in_list(self._signals, signal=s)
    357         for ref in list(self.callbacks.get(s, {}).values()):
    358             func = ref()

```

```
/usr/local/lib/python3.11/dist-packages/matplotlib/_api/__init__.py in check_in_list(values,
_print_supported_values, **kwargs)
    123     if not kwargs:
    124         raise TypeError("No argument to check!")
--> 125     for key, val in kwargs.items():
    126         if val not in values:
    127             msg = f"{val!r} is not a valid value for {key}"
```

KeyboardInterrupt:

, ¿qué clúster tiene más datos? ¿Cuántos clústeres recomiendas generar y por qué? Es decir, k=?.**